

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE SOFTWARE

INFORME TÉCNICO

PRÁCTICA EXPERIMENTAL — UNIDAD II

**Herramientas de Programación Web del Servidor:
Aplicación Dinámica con PHP 8 y ASP.NET Core / Java**

Fundamento Teórico

Asignatura: Aplicaciones Web

Nivel: 5to Nivel – A

Docente: Ing. Guerrero Ulloa Gleiston Cicerón

Periodo académico: 2026–2027 PPA

Integrantes:

Moncayo Loor Xavier Alejandro

Umaginga Arevalo Jefferson Manuel

Zambrano Moreira Alison Ariana

Repositorio del proyecto:

https:

`//github.com/a1l2i3s4o5n6/GA----Pr-ctica-Experimental-Unidad-II--.git`

Quevedo – Los Ríos – Ecuador

Índice

1. Introducción	2
2. Objetivos	2
2.1. Objetivo general	2
2.2. Objetivos específicos	3
3. Revisión Bibliográfica	3
Referencias	4
4. Fundamento Teórico	5
4.1. Ciclo de vida del servidor web y arquitecturas de integración	5
4.2. Autenticación segura y gestión de sesiones	7
4.3. Acceso a datos seguro: PDO y prepared statements	9
4.4. ASP.NET Core y Java/Spring Boot: arquitectura y seguridad	11
5. Tabla Comparativa de Tecnologías	13
6. Resultados	15
6.1. Registro de usuarios	15
6.2. Inicio de sesión	15
6.3. Dashboard protegido	16
6.4. Módulo CRUD	17
6.5. Evidencia de controles OWASP	19
6.5.1. Token CSRF	19
6.5.2. Almacenamiento seguro de contraseñas	20
6.5.3. Cabeceras HTTP de seguridad	20
6.5.4. Validación y saneamiento contra XSS	21
6.6. Evidencia de commits	21
7. Conclusiones	23
Referencias	25

1. Introducción

La presente práctica experimental se enmarca en la Unidad II de la asignatura Aplicaciones Web, correspondiente al quinto nivel de la carrera de Software de la Facultad de Ciencias de la Computación de la Universidad Técnica Estatal de Quevedo (UTEQ). Su propósito es consolidar, desde una perspectiva teórica rigurosa, los fundamentos de la programación web del lado del servidor, comparando las arquitecturas de integración servidor-lenguaje disponibles —desde el modelo histórico CGI/PERL hasta los servidores embebidos modernos de ASP.NET Core y Java/Spring Boot— y profundizando en los mecanismos de seguridad que toda aplicación web dinámica debe implementar conforme al marco de referencia OWASP Top 10:2021.

Este informe desarrolla el fundamento teórico correspondiente a los cuatro subtemas establecidos en la guía de práctica: el ciclo de vida del servidor web y los modelos de integración; la autenticación segura y la gestión de sesiones; el acceso a datos protegido contra inyección SQL mediante sentencias preparadas; y la arquitectura de seguridad de ASP.NET Core y Spring Boot. Cada subtema combina la descripción conceptual de los mecanismos involucrados con fragmentos de código representativos y comparaciones objetivas entre PHP 8.x, ASP.NET Core y Java/Spring Boot, sustentadas en fuentes bibliográficas de acceso libre y gratuito, citadas en formato IEEE.

El informe culmina con una tabla comparativa de criterios técnicos y un conjunto de conclusiones que sintetizan los principales hallazgos teóricos, sirviendo como base conceptual para la posterior fase de implementación práctica del Proyecto Fin de Curso (PFC).

2. Objetivos

2.1. Objetivo general

Desarrollar el fundamento teórico necesario para implementar el backend del Proyecto Fin de Curso con al menos dos tecnologías de programación del lado del servidor —PHP 8.x y ASP.NET Core o Java/Spring Boot—, abordando la autenticación segura, el acceso a datos mediante sentencias preparadas y las medidas de seguridad OWASP correspondientes, y estableciendo criterios técnicos objetivos para la comparación de ambas tecnologías.

2.2. Objetivos específicos

- Analizar los modelos de integración servidor-lenguaje (CGI, módulos embebidos, FastCGI y servidores embebidos modernos) y su impacto en el rendimiento y la escalabilidad.
- Describir los mecanismos de almacenamiento seguro de contraseñas y de gestión de sesiones que mitigan las vulnerabilidades OWASP A01, A02 y A07.
- Explicar el funcionamiento de las sentencias preparadas como mecanismo de prevención de inyección SQL y justificar el uso del patrón Repository.
- Comparar la arquitectura de middleware e inyección de dependencias de ASP.NET Core con el contenedor de Inversión de Control y Spring Security de Spring Boot, en relación con los principios SOLID.

3. Revisión Bibliográfica

La programación web del lado del servidor constituye uno de los pilares fundamentales en el desarrollo de aplicaciones web modernas, ya que permite procesar solicitudes, gestionar usuarios, controlar el acceso a recursos y administrar la interacción con bases de datos. Para el desarrollo de la presente práctica experimental se realizó una revisión bibliográfica basada en fuentes académicas, libros especializados y documentación técnica oficial relacionada con PHP, ASP.NET Core y Java/Spring Boot.

De acuerdo con Niederst Robbins (2024), las tecnologías web actuales han evolucionado hacia arquitecturas más robustas que permiten separar la lógica de negocio, la capa de presentación y el acceso a datos, facilitando el mantenimiento y la escalabilidad de las aplicaciones. En este contexto, los lenguajes de programación del lado del servidor desempeñan un papel fundamental en la construcción de sistemas dinámicos y seguros.

Según la documentación oficial de PHP (The PHP Group, 2026), este lenguaje continúa siendo una de las tecnologías más utilizadas para el desarrollo web debido a su facilidad de aprendizaje, amplia compatibilidad con servidores y soporte para mecanismos de seguridad como el almacenamiento seguro de contraseñas mediante `password_hash()` y el acceso a bases de datos utilizando PDO y sentencias preparadas.

Por otra parte, Microsoft (2026) destaca que ASP.NET Core proporciona un entorno moderno para el desarrollo de aplicaciones web empresariales, incorporando de manera nativa mecanismos de autenticación, autorización, inyección de dependencias y protección contra ataques comunes. Su arquitectura basada en middleware permite construir aplicaciones escalables y seguras siguiendo buenas prácticas de ingeniería de software.

En el ecosistema Java, Spring Boot se ha consolidado como uno de los frameworks más utilizados para el desarrollo de aplicaciones empresariales. De acuerdo con Walls (2022), Spring Boot simplifica la configuración de aplicaciones Java y facilita la implementación de servicios web, seguridad, acceso a datos y arquitecturas basadas en microservicios mediante la integración con Spring Security y Spring Data.

La seguridad constituye un aspecto crítico en cualquier aplicación web. El proyecto OWASP (2021) identifica vulnerabilidades frecuentes como la inyección SQL, las fallas de autenticación, el Cross-Site Scripting (XSS) y los problemas de control de acceso. Por ello, recomienda la implementación de mecanismos como sentencias preparadas, validación de entradas, tokens CSRF, almacenamiento seguro de contraseñas y cabeceras HTTP de seguridad.

Finalmente, Pressman y Maxim (2020) señalan que la aplicación de principios de diseño de software, patrones arquitectónicos y buenas prácticas de desarrollo contribuye significativamente a la calidad, mantenibilidad y seguridad de los sistemas informáticos, especialmente en aplicaciones web que manejan información sensible de los usuarios.

La revisión bibliográfica realizada proporciona el fundamento conceptual necesario para comprender las tecnologías analizadas y justificar las decisiones técnicas empleadas en el desarrollo de aplicaciones web seguras y escalables.

Referencias

N°	Referencia APA
1	Microsoft. (2026). <i>ASP.NET Core documentation</i> . Recuperado de https://learn.microsoft.com/aspnet/core
2	Niederst Robbins, J. (2024). <i>Learning Web Design</i> (6th ed.). O'Reilly Media.

3	OWASP Foundation. (2021). <i>OWASP Top 10: The ten most critical web application security risks</i> . Recuperado de https://owasp.org/www-project-top-ten/
4	Oracle Corporation. (2026). <i>Java Documentation</i> . Recuperado de https://docs.oracle.com/en/java/
5	Pressman, R. S., & Maxim, B. R. (2020). <i>Software Engineering: A Practitioner's Approach</i> (9th ed.). McGraw-Hill Education.
6	The PHP Group. (2026). <i>PHP Manual</i> . Recuperado de https://www.php.net/docs.php
7	Walls, C. (2022). <i>Spring Boot in Action</i> (2nd ed.). Manning Publications.

4. Fundamento Teórico

4.1. Ciclo de vida del servidor web y arquitecturas de integración

El procesamiento de una solicitud HTTP en el lado del servidor depende del modelo de integración entre el servidor web y el lenguaje de programación elegido para generar contenido dinámico. Históricamente, el primer mecanismo estandarizado fue el Common Gateway Interface (CGI), un protocolo que define cómo un servidor web invoca un programa externo, le pasa los datos de la petición a través de variables de entorno y del flujo estándar de entrada (*stdin*), y recibe la respuesta por la salida estándar (*stdout*) junto con las cabeceras HTTP correspondientes [2].

El modelo CGI clásico crea un proceso del sistema operativo completamente nuevo por cada solicitud entrante; este proceso carga el intérprete, ejecuta el script y termina, sin conservar estado ni memoria entre peticiones. Este modelo “un proceso por solicitud” simplifica el aislamiento entre peticiones, pero introduce un costo de arranque (*fork/exec*) que limita gravemente el rendimiento bajo carga, ya que cada solicitud paga el precio completo de inicializar el entorno de ejecución [3].

Para superar esta limitación surgieron los módulos embebidos en el propio servidor, como `mod_php` para Apache, en los cuales el intérprete del lenguaje se carga una sola vez dentro del proceso del servidor web y permanece residente en memoria, atendiendo múltiples solicitudes sin reiniciarse en cada una. Esto reduce drásticamente la latencia,

pero acopla el ciclo de vida del lenguaje al del servidor y complica el aislamiento de memoria entre peticiones concurrentes [3].

Una solución intermedia, ampliamente adoptada por el ecosistema PHP moderno, es FastCGI, implementado a través de PHP-FPM (*FastCGI Process Manager*): un conjunto configurable de procesos trabajadores persistentes (*worker pool*) se mantiene en ejecución y el servidor web (Nginx o Apache) les reenvía las solicitudes mediante un protocolo binario eficiente, amortizando el costo de arranque y permitiendo escalar horizontalmente el número de *workers* según la carga [1].

Las tecnologías más recientes evolucionan hacia servidores embebidos dentro de la propia aplicación. ASP.NET Core utiliza Kestrel, un servidor HTTP multiplataforma gestionado íntegramente por el *runtime* de .NET, normalmente colocado detrás de un proxy inverso (IIS, Nginx) en producción [4]. De manera análoga, las aplicaciones Spring Boot embeben un contenedor de servlets —típicamente Apache Tomcat— dentro del propio artefacto ejecutable (.jar), eliminando la necesidad de desplegar un servidor de aplicaciones externo. Ambos modelos comparten la filosofía de “aplicación como servidor”: el ciclo de vida del proceso HTTP coincide con el de la aplicación, lo que favorece la contenedorización con Docker y el despliegue en la nube [5].

En cuanto a PERL y el protocolo CGI, su relevancia ha decrecido sustancialmente en el desarrollo web moderno debido a las limitaciones de rendimiento descritas y a la ausencia de un ecosistema de *frameworks* web competitivo frente a PHP o Node.js. No obstante, PERL conserva nichos donde sigue siendo valioso en la actualidad: la administración de sistemas Unix/Linux, por su disponibilidad nativa y sus potentes capacidades de procesamiento de texto con expresiones regulares; la bioinformática, donde bibliotecas históricas como BioPerl continúan vigentes en *pipelines* de análisis genómico; y el *scripting* de procesamiento por lotes de archivos de texto y registros (*logs*) de servidor [2].

La elección de la tecnología de servidor para un proyecto académico o profesional debe considerar criterios objetivos como la curva de aprendizaje, la disponibilidad de *hosting* compartido económico —criterio especialmente relevante en el mercado ecuatoriano, donde el *hosting* PHP/MySQL es predominante y de bajo costo—, el tamaño de la comunidad, el ecosistema de paquetes (Composer, NuGet, Maven), el rendimiento bajo *benchmarks* reproducibles, la disponibilidad de desarrolladores en el mercado local, la facilidad de integración con proveedores de nube, el modelo de licenciamiento y la existencia de soporte

a largo plazo (LTS) del lenguaje y su entorno de ejecución. Estos criterios se desarrollan de forma comparativa en la sección 4 de este informe.

4.2. Autenticación segura y gestión de sesiones

El almacenamiento de contraseñas en texto plano constituye una de las fallas de seguridad más graves y, lamentablemente, más recurrentes en aplicaciones web; ante una fuga de base de datos, expone de inmediato las credenciales de todos los usuarios. Algoritmos de hash de propósito general como MD5 o SHA-1 tampoco son adecuados para este fin: fueron diseñados para ser extremadamente rápidos, propiedad deseable en la verificación de integridad de archivos, pero contraproducente en la protección de contraseñas, pues permite a un atacante con hardware especializado (GPU o FPGA) probar miles de millones de combinaciones por segundo mediante fuerza bruta o diccionario [6].

Los algoritmos diseñados específicamente para el hash de contraseñas —bcrypt, Argon2i y Argon2id— introducen deliberadamente un factor de costo (*work factor*) configurable que ralentiza cada intento de verificación. Bcrypt, basado en el cifrado Blowfish, permite ajustar un factor de costo logarítmico, pero su huella de memoria es relativamente pequeña y fija, lo que reduce su resistencia frente a ataques paralelizados en GPU. Argon2, ganador de la Password Hashing Competition, corrige esta debilidad permitiendo configurar de forma independiente el costo de memoria, el costo de tiempo y el grado de paralelismo; su variante Argon2id combina resistencia a ataques de canal lateral con resistencia a ataques de compromiso tiempo–memoria, por lo que es actualmente la recomendación por defecto de OWASP.

PHP 8.x materializa esta recomendación mediante la función `password_hash()`, que admite las constantes `PASSWORD_BCRYPT` y `PASSWORD_ARGON2ID`, delegando en el propio lenguaje la generación de la sal aleatoria y el formato del hash [1]. En el ecosistema .NET, ASP.NET Core Identity utiliza por defecto PBKDF2 con HMAC-SHA256, aplicando un número elevado y configurable de iteraciones como mecanismo de costo computacional [4].

Ejemplo en PHP 8.x: Registro y verificación con Argon2id

Cuadro 2: Ejemplo de registro y verificación de contraseñas con Argon2id en PHP 8.x

```
$hash = password_hash($password, PASSWORD_ARGON2ID);  
  
// ...  
  
if (password_verify($claveIngresada, $hash) {  
    session_regenerate_id(true); // mitiga session fixation  
}
```

Un aspecto frecuentemente pasado por alto es el ataque de temporización (*timing attack*): si la comparación entre el hash almacenado y el calculado se realiza con una función de comparación de cadenas convencional, esta suele detenerse en el primer byte distinto, filtrando información sobre cuántos caracteres iniciales coinciden. Tanto `password_verify()` en PHP como `PasswordHasher.VerifyHashedPassword()` en ASP.NET Core mitigan este riesgo mediante comparaciones de tiempo constante, que recorren la totalidad de la cadena sin importar en qué posición ocurre la primera discrepancia [6].

La gestión de sesiones presenta riesgos igualmente críticos. La fijación de sesión (*session fixation*) ocurre cuando un atacante logra que la víctima utilice un identificador de sesión conocido de antemano por él; si la aplicación no genera un nuevo identificador tras la autenticación exitosa, el atacante puede reutilizar ese mismo identificador para suplantar a la víctima ya autenticada. La contramedida estándar en PHP es invocar `session_regenerate_id(true)` inmediatamente después de validar las credenciales [1].

El secuestro de sesión (*session hijacking*), por su parte, ocurre cuando el atacante intercepta o roba un identificador válido, habitualmente mediante *scripting* entre sitios (XSS) o interceptación de tráfico no cifrado; las cabeceras de cookies `HttpOnly`, `Secure` y `SameSite=Strict` constituyen las contramedidas estándar a nivel de navegador.

En aplicaciones distribuidas, un almacén centralizado como Redis permite compartir el estado de sesión entre instancias con baja latencia, a diferencia del almacenamiento en archivos del sistema de archivos local. Como alternativa al modelo de sesión con estado, los tokens JWT ofrecen un esquema sin estado en el cual toda la información viaja firmada dentro del propio token: favorecen la escalabilidad horizontal, pero complican la revocación anticipada de credenciales y exponen su contenido, dado que el *payload* solo está codificado

en Base64 —no cifrado—.

Trasladando estos conceptos al Top 10 de OWASP 2021, el módulo de autenticación de cualquier proyecto académico debe abordar explícitamente A07 (Fallas de Identificación y Autenticación) mediante hash robusto de contraseñas y regeneración de sesión; A02 (Fallas Criptográficas), evitando algoritmos obsoletos y garantizando el transporte cifrado mediante HTTPS; y A01 (Control de Acceso Roto), verificando en cada ruta protegida que exista una sesión autenticada antes de servir el recurso, en lugar de confiar únicamente en la ausencia de enlaces visibles hacia esa funcionalidad.

4.3. Acceso a datos seguro: PDO y prepared statements

La inyección SQL sigue figurando entre las vulnerabilidades más explotadas en aplicaciones web pese a ser, conceptualmente, un problema resuelto desde hace más de dos décadas. Su mecanismo básico consiste en que una entrada proporcionada por el usuario se concatena directamente dentro de una sentencia SQL sin validación ni escape, permitiendo que esa entrada altere la estructura lógica de la consulta. El ejemplo clásico de evasión de autenticación (*login bypass*) lo ilustra con claridad: si una consulta se construye concatenando directamente los campos de usuario y clave, y el atacante introduce `' OR '1'='1` en cualquiera de ellos, la condición resultante se vuelve siempre verdadera, devolviendo el primer registro de la tabla sin necesidad de conocer ninguna credencial válida [7].

Las consecuencias de una inyección SQL no autorizada van mucho más allá de la evasión de un formulario de inicio de sesión: incluyen la extracción completa de la base de datos, la modificación o eliminación de registros y, en motores que lo permiten, la ejecución de comandos del sistema operativo subyacente. Bajo el sistema de puntuación CVSS v3.1, una inyección SQL que comprometa confidencialidad, integridad y disponibilidad sin requerir privilegios ni interacción del usuario suele clasificarse en el rango crítico (9.0–10.0) [7].

La defensa estándar contra la inyección SQL son las sentencias preparadas (*prepared statements*), implementadas en PHP mediante la extensión PDO. El mecanismo opera en dos fases separadas: en `prepare()`, el motor de base de datos recibe la plantilla de la consulta con marcadores de posición en lugar de valores literales y compila un plan de ejecución para esa estructura; en `execute()`, los valores reales se envían por separado, a

través de un canal distinto del de la sentencia SQL. Como el plan de consulta ya quedó fijado en la fase de preparación, ningún valor enviado después puede alterar la estructura sintáctica de la sentencia [1].

PDO ofrece dos métodos para vincular parámetros: `bindParam()`, que vincula por referencia y refleja el valor de la variable en el momento de `execute()` —no en el momento de `bindParam()`—, y `bindValue()`, que vincula por valor, capturando el contenido inmediatamente; esta distinción resulta crítica en bucles donde la variable cambia entre la vinculación y la ejecución.

Cuadro 3: CRUD seguro con PDO en PHP 8.x

```
$stmt = $pdo->prepare(
    'SELECT id, usuario FROM usuarios WHERE usuario =
:usuario'
);

$stmt->bindValue(':usuario', $usuario, PDO::PARAM_STR);

try {
    $stmt->execute();
    $row = $stmt->fetch(PDO::FETCH_ASSOC);
} catch (PDOException $e) {
    // registrar internamente;
    // nunca exponer $e->getMessage() al usuario
}
```

El patrón Repository desacopla la lógica de acceso a datos de la lógica de negocio mediante una capa de abstracción: una interfaz (por ejemplo, `UsuarioRepositoryInterface`) declara los métodos de persistencia sin especificar su implementación, mientras que una clase concreta (`PdoUsuarioRepository`) la implementa utilizando PDO. Este desacoplamiento, alineado con el Principio de Inversión de Dependencias (DIP) del conjunto SOLID, permite sustituir la implementación de persistencia —por ejemplo, por un repositorio en memoria para pruebas unitarias— sin modificar el código de los servicios que dependen de la interfaz [10].

Comparado con el patrón Active Record propio de ORMs como Eloquent, donde cada entidad sabe guardarse y recuperarse a sí misma, el patrón Repository añade valor

real cuando el proyecto requiere pruebas automatizadas exhaustivas, múltiples fuentes de datos intercambiables o una separación arquitectónica estricta entre capas; en proyectos pequeños sin estos requisitos, la indirección adicional puede constituir sobreingeniería que ralentiza el desarrollo sin un beneficio proporcional.

Al comparar el acceso a datos parametrizado entre tecnologías, ADO.NET en C# expone la clase `SqlCommand` junto con la colección `Parameters`, donde cada parámetro se añade explícitamente mediante `SqlParameter` antes de ejecutar la consulta, con un *pool* de conexiones gestionado de forma automática por el proveedor de datos. En Java, JDBC ofrece un mecanismo equivalente mediante `PreparedStatement`, donde los marcadores de posición (?) se vinculan por índice posicional con métodos como `setString()` o `setInt()` [8], [9].

Java tiende a requerir más código repetitivo para el manejo manual de conexiones y excepciones —mitigado parcialmente por *try-with-resources*—, mientras que frameworks como Spring Data JPA reducen esa verbosidad mediante abstracciones de más alto nivel sobre JDBC. ““

4.4. ASP.NET Core y Java/Spring Boot: arquitectura y seguridad

ASP.NET Core organiza el procesamiento de cada solicitud HTTP mediante un *pipeline* de middleware: una secuencia de componentes de software, cada uno de los cuales recibe la solicitud, puede actuar sobre ella y decide si invoca al siguiente componente de la cadena o corta el procesamiento devolviendo una respuesta. Este diseño materializa el patrón *Chain of Responsibility*, en el cual múltiples manejadores tienen la oportunidad de procesar una solicitud sin que el emisor conozca cuál de ellos la atenderá finalmente [4].

El orden de registro del middleware es semánticamente significativo: la secuencia recomendada `UseHttpsRedirection` → `UseStaticFiles` → `UseRouting` → `UseAuthentication` → `UseAuthorization` → `MapControllers` refleja dependencias lógicas estrictas. La autenticación debe registrarse antes que la autorización por una razón evidente: la autorización responde a “¿tiene este usuario permiso para esta acción?”, pregunta que solo puede responderse si previamente se determinó quién es el usuario. Invertir este orden provocaría que las comprobaciones de autorización se ejecuten sobre un contexto de usuario aún no resuelto [5].

Cuadro 4: Orden correcto del pipeline de middleware en ASP.NET Core

```
app.UseHttpsRedirection();  
app.UseStaticFiles();  
app.UseRouting();  
app.UseAuthentication();  
app.UseAuthorization();  
app.MapControllers();
```

La inyección de dependencias (*Dependency Injection*, DI) es un mecanismo central del framework, integrado de manera nativa en el contenedor de servicios de ASP.NET Core, que gestiona tres ciclos de vida (*lifetimes*) diferenciados. Una instancia *Transient* se crea de nuevo cada vez que se solicita, adecuada para servicios ligeros y sin estado. Una instancia *Scoped* se crea una sola vez por solicitud HTTP y se reutiliza durante toda esa solicitud, siendo el *lifetime* recomendado para repositorios y contextos de acceso a datos. Una instancia *Singleton* se crea una única vez durante todo el ciclo de vida de la aplicación y se comparte entre todas las solicitudes.

Es importante evitar la inyección de servicios *Scoped* dentro de servicios *Singleton*, ya que esto provoca una captura de dependencia y comportamientos inconsistentes entre solicitudes. El repositorio de un proyecto típico debe registrarse como *Scoped*, equilibrando rendimiento y consistencia de los datos [4].

Spring Boot resuelve un problema equivalente mediante su contenedor de Inversión de Control (*IoC*), que comparte con ASP.NET Core el principio general de inversión de control —el framework, y no el código de la aplicación, decide cuándo y cómo se instancian las dependencias— pero diverge en la filosofía de configuración. Mientras ASP.NET Core favorece el registro explícito de servicios en código, Spring se apoya fuertemente en anotaciones declarativas como `@Component`, `@Service`, `@Repository` y `@Autowired`, combinadas con el principio de “convención sobre configuración” [9].

Spring Security se integra como una cadena de filtros (*SecurityFilterChain*) que intercepta cada solicitud antes de que llegue al controlador, de forma conceptualmente análoga al *pipeline* de middleware de ASP.NET Core. El *AuthenticationManager* centraliza la verificación de credenciales, delegando en una o más implementaciones de *UserDetailsService* la recuperación de la información del usuario desde la fuente de datos correspondiente, lo

que permite intercambiar el origen de las credenciales sin modificar el resto de la cadena de seguridad [9].

Finalmente, los cinco principios SOLID —Responsabilidad Única (SRP), Abierto/Cerrado (OCP), Sustitución de Liskov (LSP), Segregación de Interfaces (ISP) e Inversión de Dependencias (DIP)— proporcionan un marco objetivo para evaluar la calidad arquitectónica del backend de cualquier proyecto.

El SRP, por ejemplo, se viola cuando una misma clase de repositorio asume simultáneamente la persistencia de datos y el envío de notificaciones por correo; corregirlo implica extraer un servicio de notificaciones independiente que el controlador orqueste junto al repositorio. El DIP, ya mencionado en relación con el patrón Repository, se viola cuando un controlador instancia directamente una clase concreta de acceso a datos en lugar de recibir una abstracción inyectada, acoplando fuertemente las capas superiores a los detalles de implementación de la persistencia y dificultando las pruebas unitarias [10].

5. Tabla Comparativa de Tecnologías

La siguiente tabla resume criterios técnicos objetivos para comparar PHP 8.x, ASP.NET Core y Java/Spring Boot como tecnologías de servidor, sintetizando los aspectos desarrollados en la sección anterior y sirviendo de base para la justificación tecnológica del Proyecto Fin de Curso.

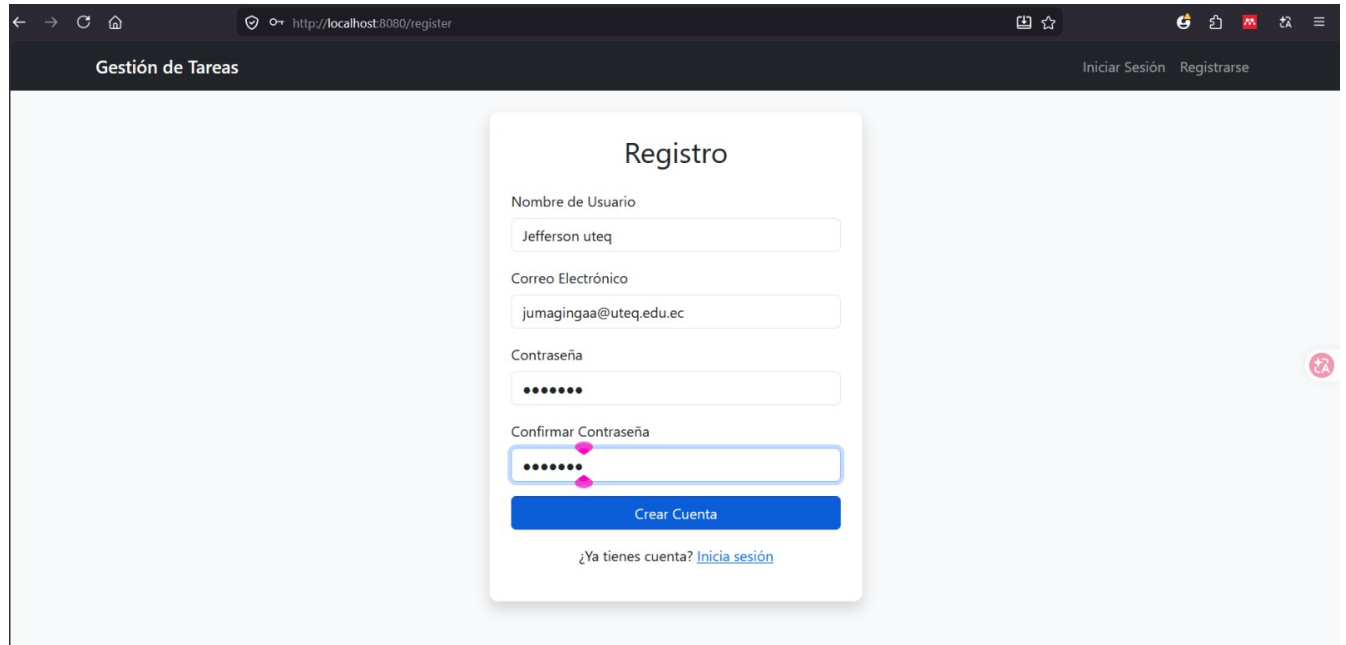
Cuadro 5: Comparación de tecnologías de programación web del lado del servidor

Criterio técnico	PHP 8.x (PDO + PHP-FPM)	ASP.NET Core 8 (C#)	Java 21 + Spring Boot 3
Curva de aprendizaje	Baja: sintaxis simple y abundante documentación en español.	Media-alta: requiere C# y conocimientos sólidos de POO.	Alta: POO estricta, anotaciones y configuración del contenedor IoC.
Seguridad por defecto	Media: hash, CSRF y cabeceras se configuran manualmente.	Alta: Identity, antiforgery tokens y cabeceras integradas.	Alta: Spring Security integrado con configuración declarativa.
Rendimiento	Medio-alto con OPcache y JIT (PHP 8.0+).	Muy alto; Kestrel figura entre los más rápidos en benchmarks.	Alto; Tomcat embebido, mejorable con WebFlux reactivo.
Ecosistema	Composer / Packagist, muy amplio.	NuGet, amplio y respaldado por Microsoft.	Maven Central, uno de los ecosistemas más maduros y extensos.
Hosting compartido en Ecuador / costo	Muy disponible y económico (cPanel estándar).	Limitado; suele requerir hosting Windows o contenedores.	Limitado; generalmente requiere VPS o contenedores.
Comunidad y soporte en español	Muy amplia.	Moderada.	Amplia.
Tipado del lenguaje	Dinámico, con tipado opcional desde PHP 7.	Estático fuerte (C#).	Estático fuerte (Java).
Concurrencia y escalabilidad	Basada en pool de procesos (PHP-FPM).	<i>async/await</i> nativo; alta concurrencia con pocos hilos.	Hilos gestionados por el contenedor; soporte reactivo opcional.

6. Resultados

6.1. Registro de usuarios

La Figura 1 muestra el formulario de registro implementado para la creación de nuevas cuentas de usuario.



The screenshot displays a web browser window with the address bar showing `http://localhost:8080/register`. The page has a dark header with the text "Gestión de Tareas" on the left and "Iniciar Sesión" and "Registrarse" on the right. The main content area features a white registration form titled "Registro". The form includes the following fields: "Nombre de Usuario" with the value "Jefferson uteq", "Correo Electrónico" with the value "jumagingaa@uteq.edu.ec", "Contraseña" (password) masked with dots, and "Confirmar Contraseña" (confirm password) also masked with dots. A blue "Crear Cuenta" button is positioned below the password fields. At the bottom of the form, there is a link that reads "¿Ya tienes cuenta? [Inicia sesión](#)".

Figura 1: Formulario de registro de usuarios.

6.2. Inicio de sesión

La Figura 2 presenta la interfaz de autenticación utilizada para validar las credenciales de acceso.

The image shows a web browser window with the address bar displaying `http://localhost:8080/login`. The page has a dark header with the text "Gestión de Tareas" and a small square icon on the right. The main content area is light gray and contains a white login form. The form is titled "Iniciar Sesión" and includes two input fields: "Correo Electrónico" and "Contraseña". Below these fields is a blue button labeled "Ingresar". At the bottom of the form, there is a link that says "¿No tienes cuenta? [Regístrate aquí](#)".

Figura 2: Formulario de inicio de sesión.

6.3. Dashboard protegido

La Figura 3 muestra las principales vistas del dashboard protegido, accesibles únicamente después de una autenticación exitosa.

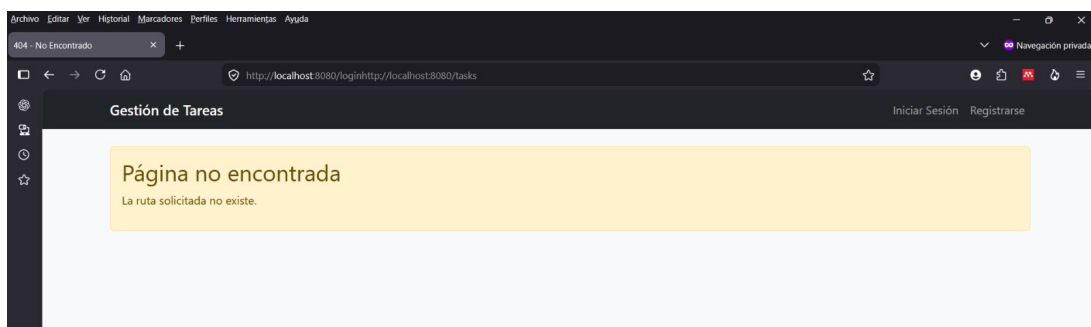
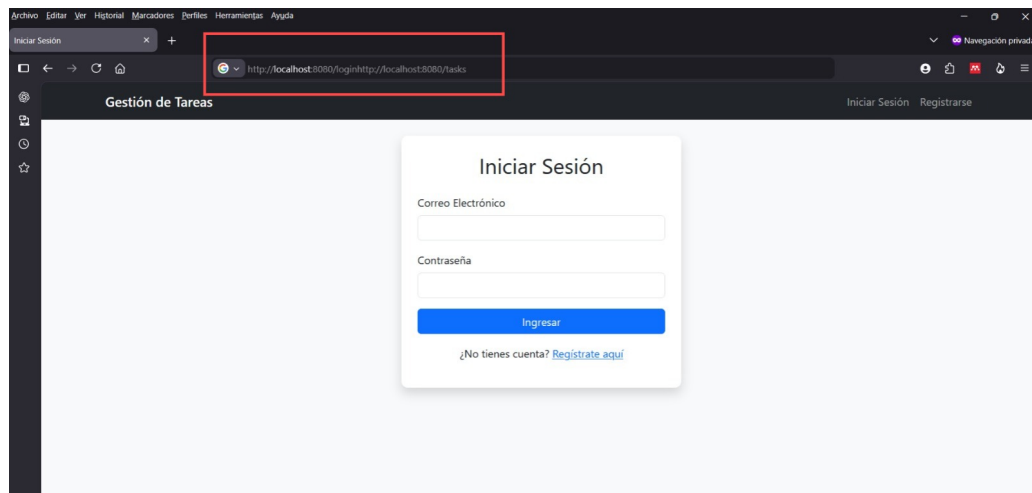


Figura 3: Dashboard protegido tras la autenticación del usuario.

6.4. Módulo CRUD

Las Figuras 4, 5, 6 y 7 evidencian las operaciones CRUD implementadas en el sistema.

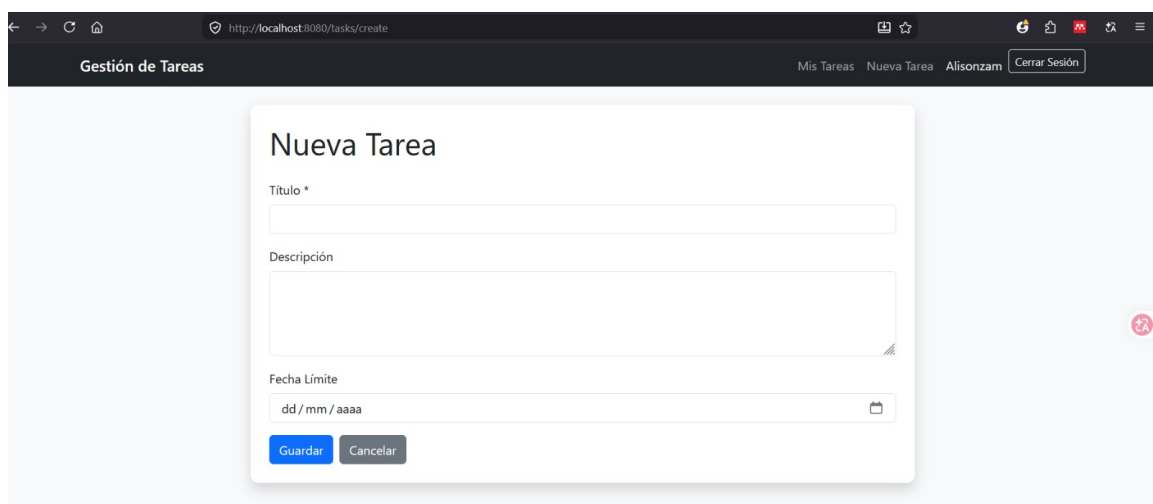


Figura 4: Operación Crear (Create).

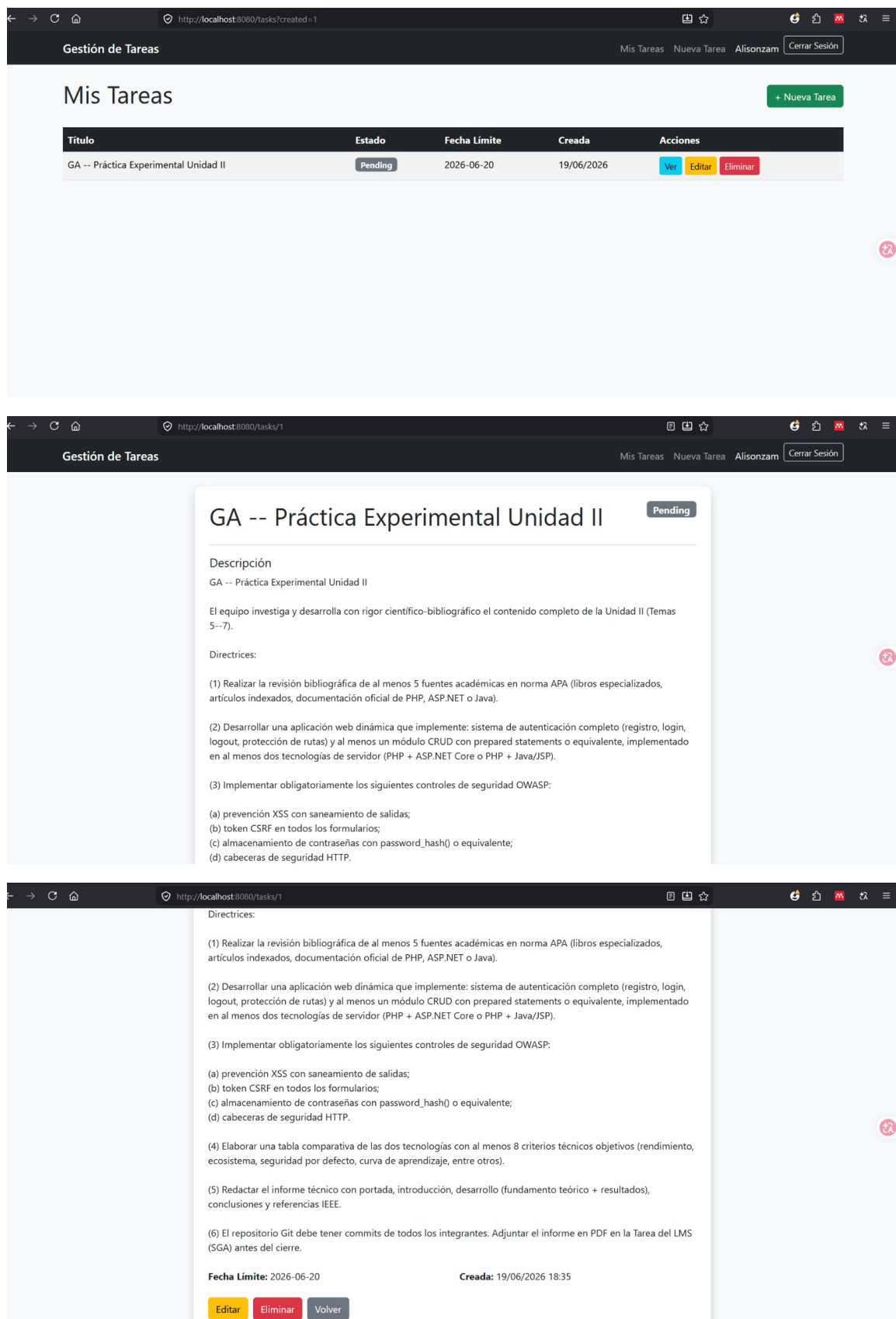


Figura 5: Operación Consultar (Read).

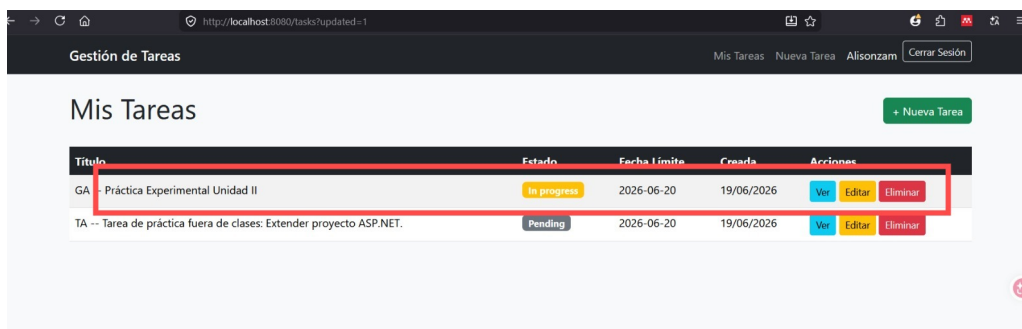


Figura 6: Operación Editar (Update).

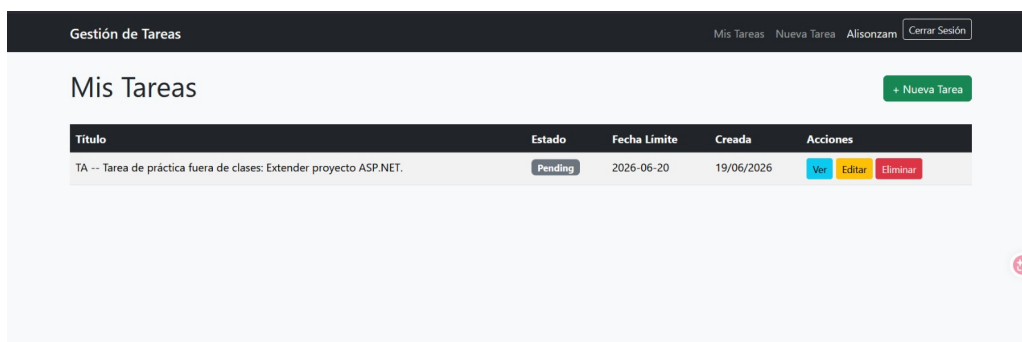
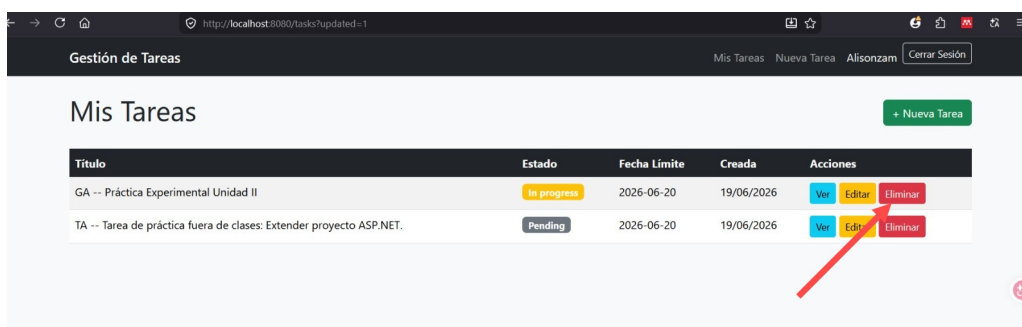


Figura 7: Operación Eliminar (Delete).

6.5. Evidencia de controles OWASP

Con el fin de demostrar la implementación de controles de seguridad alineados con las recomendaciones de OWASP Top 10:2021, se presentan las siguientes evidencias.

6.5.1. Token CSRF

La Figura 8 muestra la generación e inclusión de un token CSRF en los formularios del sistema para prevenir ataques de falsificación de solicitudes entre sitios.

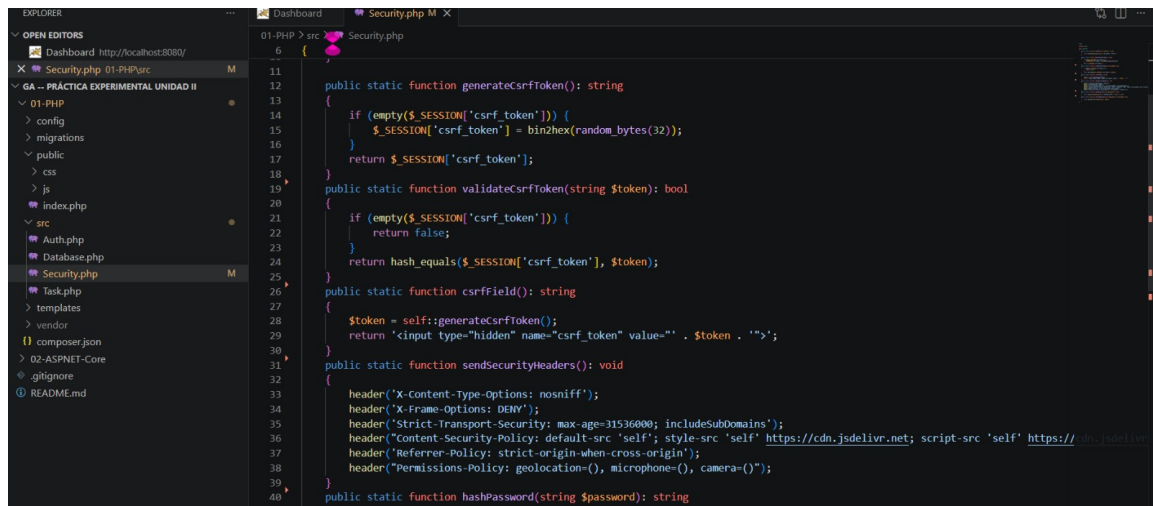


Figura 8: Implementación de token CSRF.

6.5.2. Almacenamiento seguro de contraseñas

La Figura 9 evidencia que las contraseñas no se almacenan en texto plano, sino mediante funciones de hash seguras.

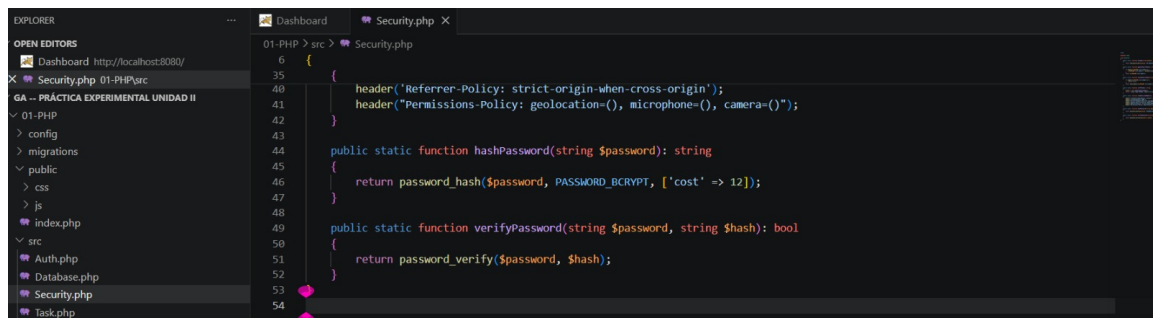


Figura 9: Contraseña almacenada mediante hash seguro.

6.5.3. Cabeceras HTTP de seguridad

La Figura 10 muestra las cabeceras HTTP configuradas para reforzar la seguridad de la aplicación.

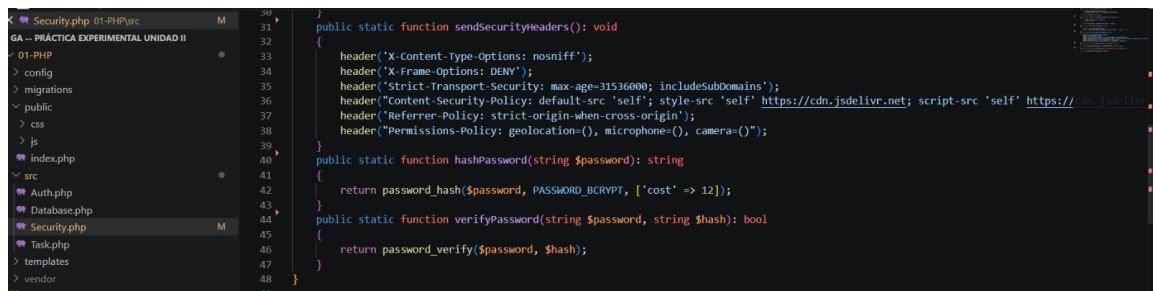


Figura 10: Cabeceras HTTP de seguridad.

6.5.4. Validación y saneamiento contra XSS

La Figura 11 evidencia la validación y sanitización de entradas para prevenir ataques Cross-Site Scripting (XSS).

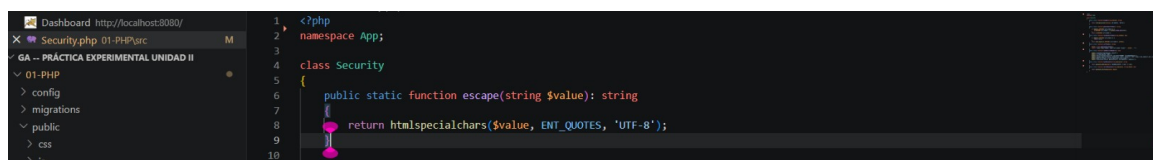
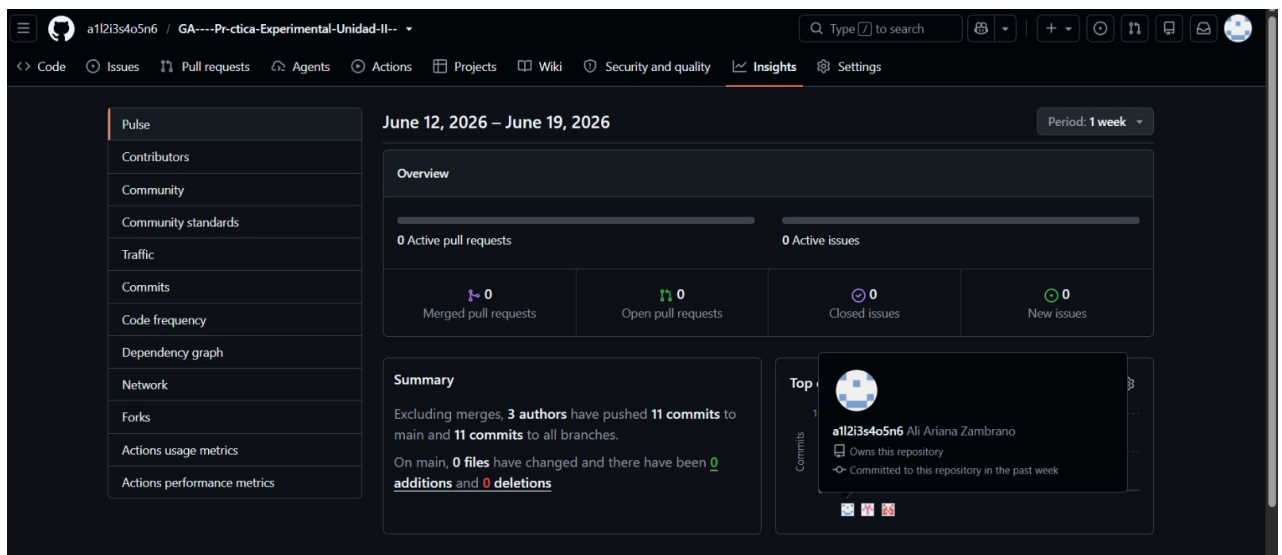


Figura 11: Protección contra ataques XSS.

6.6. Evidencia de commits

Con el fin de demostrar el uso de Git como sistema de control de versiones durante el desarrollo del proyecto, se presenta la evidencia del historial de commits realizados. Cada commit representa una modificación registrada en el repositorio, permitiendo mantener la trazabilidad de los cambios y facilitar el trabajo colaborativo entre los integrantes del equipo.



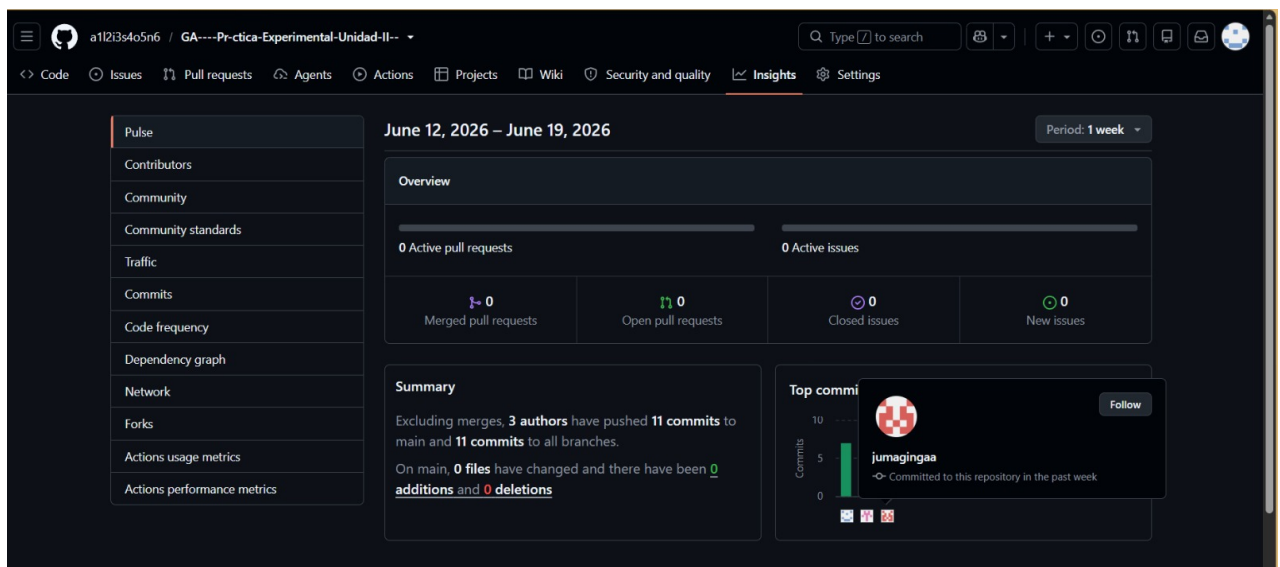
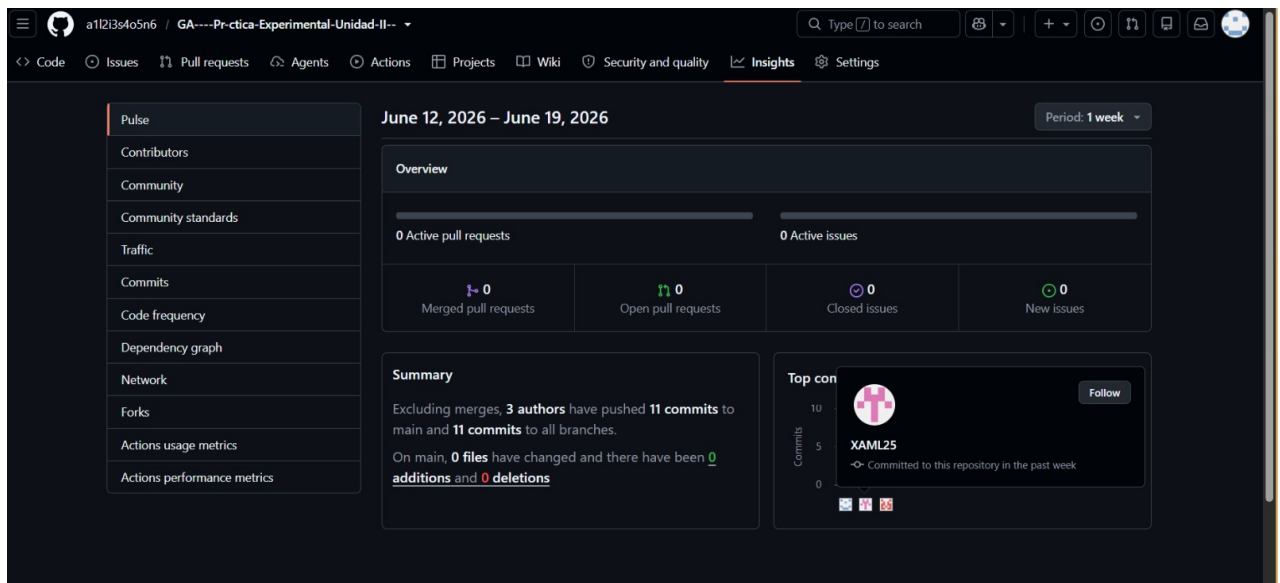


Figura 12: Evidencias del commit.

7. Conclusiones

El análisis teórico desarrollado evidencia que la elección de una tecnología de servidor no admite una respuesta universal, sino que depende de un balance entre criterios técnicos objetivos y el contexto particular del proyecto. PHP 8.x, ejecutado mediante PHP-FPM, ofrece la curva de aprendizaje más baja y el ecosistema de hosting más accesible y económico en el mercado ecuatoriano, pero traslada al desarrollador buena parte de la responsabilidad de implementar controles de seguridad —hash de contraseñas, tokens CSRF y cabeceras HTTP— que ASP.NET Core y Spring Boot proveen de manera más

integrada por defecto a través de Identity y Spring Security, respectivamente [1], [4], [9].

Desde la perspectiva de la prevención de vulnerabilidades, el estudio confirma que el mecanismo de sentencias preparadas en dos fases (`prepare/execute`) resuelve de raíz el problema de la inyección SQL al separar completamente la estructura de la consulta de los datos del usuario, independientemente de si se implementa mediante PDO en PHP, ADO.NET en C# o JDBC en Java [1], [7], [8]. De manera similar, el almacenamiento de contraseñas mediante algoritmos de costo configurable (Argon2id, PBKDF2) y la regeneración del identificador de sesión tras la autenticación constituyen contramedidas técnicas equivalentes, aunque expuestas con distinto nivel de fricción para el desarrollador en cada plataforma [6].

En el plano arquitectónico, tanto el *pipeline* de middleware de ASP.NET Core como la cadena de filtros de Spring Security materializan el patrón *Chain of Responsibility*, y ambos frameworks implementan contenedores de Inversión de Control que gestionan el ciclo de vida de las dependencias de forma declarativa, reduciendo el acoplamiento y favoreciendo la aplicación de los principios SOLID, en particular la Inversión de Dependencias mediante el patrón Repository [4], [9], [10].

Para el contexto específico de la provincia de Los Ríos y de Ecuador en general —donde predomina el hosting compartido de bajo costo y existe una base más amplia de desarrolladores PHP— esta tecnología resulta la opción más viable para proyectos académicos y comerciales de pequeña y mediana escala, mientras que ASP.NET Core o Spring Boot se justifican en escenarios que demanden mayor seguridad por defecto, tipado estático y escalabilidad orientada a microservicios, criterios que deberán contrastarse empíricamente en la fase de implementación del Proyecto Fin de Curso mediante pruebas de rendimiento y análisis estático de código.

Referencias

- [1] J. R. Olivas Mendoza, *PHP Succinctly*. Morrisville, NC, USA: Syncfusion, Inc., 2017. [Online]. Disponible: <https://www.syncfusion.com/succinctly-free-ebooks/php-succinctly>
- [2] GoalKicker.com, *Perl Notes for Professionals*. GoalKicker.com Free Programming Books, 2018. [Online]. Disponible: <https://goalkicker.com/PerlBook/>

- [3] M. Clifton, *Web Servers Succinctly*. Morrisville, NC, USA: Syncfusion, Inc., 2015. [Online]. Disponible: <https://www.syncfusion.com/succinctly-free-ebooks/webservers>
- [4] S. Chiaretta and U. Lattanzi, *ASP.NET Core Succinctly*. Morrisville, NC, USA: Syncfusion, Inc., 2017. [Online]. Disponible: <https://www.syncfusion.com/succinctly-free-ebooks/asp-net-core-succinctly>
- [5] D. Strauss, *ASP.NET Core 6 Succinctly*. Morrisville, NC, USA: Syncfusion, Inc., 2022. [Online]. Disponible: <https://www.syncfusion.com/succinctly-free-ebooks/asp-net-core-6-succinctly>
- [6] S. Drapkin, *Application Security in .NET Succinctly*. Morrisville, NC, USA: Syncfusion, Inc., 2017. [Online]. Disponible: <https://www.syncfusion.com/succinctly-free-ebooks/application-security-in-net-succinctly>
- [7] GoalKicker.com, *SQL Notes for Professionals*. GoalKicker.com Free Programming Books, 2018. [Online]. Disponible: <https://goalkicker.com/SQLBook/>
- [8] C. Rose, *Java Succinctly, Part 1*. Morrisville, NC, USA: Syncfusion, Inc., 2017. [Online]. Disponible: <https://www.syncfusion.com/succinctly-free-ebooks/java-succinctly-part-1>
- [9] GoalKicker.com, *Spring Framework Notes for Professionals*. GoalKicker.com Free Programming Books, 2018. [Online]. Disponible: <https://goalkicker.com/SpringFrameworkBook/SpringFrameworkNotesForProfessionals.pdf>
- [10] G. K. Arora, *SOLID Principles Succinctly*. Morrisville, NC, USA: Syncfusion, Inc., 2016. [Online]. Disponible: <https://www.syncfusion.com/succinctly-free-ebooks/solidprinciplessuccinctly>